


```

}

// 2. Extraer el token (quitar "Bearer " del inicio)
const token = authHeader.split(' ')[1];

try {
  // 3. Verificar la firma del token con el mismo secreto que se usó para crearlo
  // Si la firma no coincide o el token expiró, jwt.verify lanza una excepción
  const payload = jwt.verify(token, process.env.JWT_SECRET);

  // 4. Adjuntar los datos del usuario al objeto req para usarlos en el controlador
  req.usuario = payload; // { id, email, nombre, iat, exp }

  // 5. Pasar al siguiente middleware o controlador
  next();

} catch (error) {
  if (error.name === 'TokenExpiredError') {
    return res.status(401).json({
      error: 'Token expirado – inicia sesión nuevamente'
    });
  }
  return res.status(401).json({
    error: 'Token inválido'
  });
}
};

module.exports = autenticar;

```

9.4 Crear las rutas de autenticación

Crear `src/routes/auth.js` con registro y login:

```

▶ src/routes/auth.js
// src/routes/auth.js
// Rutas públicas (no requieren token): /auth/register y /auth/login
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const router = express.Router();

// Base de datos temporal en memoria
// En el Paso 11 se migra a Supabase con Prisma
let usuarios = [];

// — POST /auth/register

// Registrar un nuevo usuario
router.post('/register', async (req, res) => {
  try {
    const { nombre, email, password } = req.body;

    // Validaciones básicas
    if (!email || !password) {
      return res.status(400).json({ error: 'Email y password son obligatorios' });
    }
    if (password.length < 6) {
      return res.status(400).json({ error: 'La contraseña debe tener mínimo 6 caracteres' });
    }
  }
}

```

```

    if (usuarios.find(u => u.email === email)) {
      return res.status(409).json({ error: 'Ya existe una cuenta con ese email' });
    }

    // Hashear la contraseña con bcrypt
    // El 12 son las "rondas" de encriptación — más rondas = más seguro pero más lento
    // 12 es el estándar recomendado para producción en 2026
    const hash = await bcrypt.hash(password, 12);

    const nuevoUsuario = {
      id: usuarios.length + 1,
      nombre: nombre || 'Usuario',
      email: email.toLowerCase().trim(),
      password: hash // NUNCA el password en texto plano
    };
    usuarios.push(nuevoUsuario);

    // Responder SIN incluir el password (ni el hash)
    res.status(201).json({
      mensaje: 'Cuenta creada exitosamente',
      usuario: { id: nuevoUsuario.id, nombre: nuevoUsuario.nombre, email: nuevoUsuario.email
}
    });

  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// — POST /auth/login

// Iniciar sesión y obtener token JWT
router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ error: 'Email y password son obligatorios' });
    }

    // Buscar el usuario en la base de datos
    const usuario = usuarios.find(u => u.email === email.toLowerCase().trim());

    if (!usuario) {
      // Mismo mensaje para usuario no encontrado Y contraseña incorrecta
      // (por seguridad: no revelar si el email existe o no)
      return res.status(401).json({ error: 'Credenciales incorrectas' });
    }

    // Comparar el password recibido con el hash guardado
    const passwordValido = await bcrypt.compare(password, usuario.password);

    if (!passwordValido) {
      return res.status(401).json({ error: 'Credenciales incorrectas' });
    }

    // Generar el token JWT
    const token = jwt.sign(
      { id: usuario.id, email: usuario.email, nombre: usuario.nombre }, // payload
      process.env.JWT_SECRET, // clave secreta del .env
      { expiresIn: process.env.JWT_EXPIRES_IN || '1h' } // expiración

```

```

    });

    res.json({
      token,
      tipo: 'Bearer',
      expiraEn: '1 hora',
      usuario: { id: usuario.id, nombre: usuario.nombre, email: usuario.email }
    });

    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  });

  module.exports = router;

```

9.5 Conectar auth y proteger las rutas de sesiones

Abrir `src/app.js` y agregar la ruta de auth y el middleware en sesiones:

```

▶ ACTUALIZAR app.js y sesiones.js
// src/app.js – AGREGAR estas dos líneas junto a las demás rutas:
app.use('/auth', require('./routes/auth'));

// src/routes/sesiones.js – MODIFICAR para proteger POST, PUT y DELETE
const autenticar = require('../middlewares/autenticar');

router.get  ('/',      ctrl.listar);           // Público: cualquiera puede ver sesiones
router.get ('/:id',   ctrl.obtenerUna);       // Público: ver una sesión
router.post ('/',      autenticar, ctrl.crear); // PRIVADO: requiere login
router.put ('/:id',   autenticar, ctrl.actualizar); // PRIVADO
router.delete('/:id', autenticar, ctrl.eliminar); // PRIVADO

```

9.6 Probar el flujo completo de autenticación

Con el servidor corriendo, probar en este orden en Thunder Client:

#	Petición	Body	Esperas	Por qué
1	POST /auth/register	{"nombre":"Ana","email":"ana@upds.edu","password":"mipass123"}	201 + datos del usuario	El hash bcrypt se guarda, nunca el texto plano
2	POST /auth/login	{"email":"ana@upds.edu","password":"mipass123"}	200 + token JWT	Copiar el token para el paso 3
3	POST /api/sesiones SIN token	{"titulo":"test"}	401 Unauthorized	El middleware autenticar bloquea la petición
4	POST /api/sesiones CON token	{"titulo":"Sesión con auth"}	201 Created	El token es válido, req.usuario tiene el ID del usuario

Para el paso 4, en Thunder Client agregar el header de autenticación: Auth → Bearer Token → pegar el token del paso 2.



Errores frecuentes en JWT

Error	Solución
invalid signature	JWT_SECRET cambió entre cuando se generó el token y cuando se verifica. Hacer logout + nuevo login.
jwt malformed	El token está incompleto o corrupto. Verificar que se copió completo (son 3 partes separadas por puntos).
Cannot read property 'id' of undefined	El middleware autenticar no se aplicó a la ruta o el token no se decodificó. Verificar el orden de middlewares.

✓ Al finalizar el Paso 9: **POST /auth/register crea usuario con hash. POST /auth/login devuelve token. Rutas privadas devuelven 401 sin token y 200 con token válido.**

PASO 10 Supabase + Prisma — Datos Persistentes

Tiempo estimado: 1 hora · De memoria RAM a disco permanente

🎯 Hasta ahora los datos viven en variables de JavaScript. Cuando reinicias el servidor, todo se borra. Supabase es como cambiar de una pizarra (fácil de borrar) a un cuaderno de contabilidad (permanente): todo queda registrado aunque apagues el negocio.

10.1 Configurar el proyecto en Supabase

1. Ir a supabase.com → Login with GitHub → clic en 'New Project'.
2. Name: studysync-db. Database Password: una contraseña segura (guárdala en un lugar seguro). Region: US East (N. Virginia). Clic en Create New Project.
3. Esperar 60 segundos mientras Supabase provisiona el servidor PostgreSQL.
4. Ir a Settings (ícono de engranaje, abajo a la izquierda) → Database → Connection string → seleccionar 'URI' → copiar el string completo.

La DATABASE_URL se ve así:

```
► FORMATO DE DATABASE_URL
postgresql://postgres:TuContraseña@db.XXXX.supabase.co:5432/postgres
```

Actualizar el archivo .env con la URL real:

```
► ACTUALIZAR .env
DATABASE_URL=postgresql://postgres:TuContraseña@db.XXXX.supabase.co:5432/postgres
```

10.2 Instalar y configurar Prisma

🎯 Prisma es un traductor automático entre el mundo de JavaScript y el mundo de SQL. En lugar de escribir: `SELECT * FROM sesiones WHERE id = 1`, escribes: `prisma.sesion.findUnique({where:{id:1}})`. Prisma traduce eso a SQL por ti.

```
► INSTALAR Y CONFIGURAR PRISMA
# @prisma/client: el módulo que usas en el código
# prisma: las herramientas de línea de comandos (generar, migrar, etc.)
npm install @prisma/client
npm install -D prisma

# Inicializar Prisma en el proyecto
# Crea la carpeta prisma/ con el archivo schema.prisma
npx prisma init
```

```
► RESULTADO ESPERADO
Your Prisma schema was created at prisma/schema.prisma
```

10.3 Definir el esquema de la base de datos

Abrir el archivo `prisma/schema.prisma` y reemplazar su contenido completo:

```
► prisma/schema.prisma
// prisma/schema.prisma
// Este archivo describe la estructura de la base de datos
// Es el equivalente a los CREATE TABLE de SQL, pero en un lenguaje más legible
```

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL") // Lee el .env
}

// Modelo Usuario: representa la tabla 'usuarios' en la BD
model Usuario {
  id          Int          @id @default(autoincrement()) // Clave primaria autoincremental
  nombre      String
  email       String       @unique                       // No puede haber dos con el mismo email
  password    String       // Hash de bcrypt, nunca texto plano
  creadoEn    DateTime     @default(now())               // Se llena automáticamente al crear
  sesiones    Sesión[]     // Relación: un usuario puede tener muchas sesiones
}

// Modelo Sesión: representa la tabla 'sesiones' en la BD
model Sesión {
  id          Int          @id @default(autoincrement())
  titulo       String
  descripcion  String?     // El ? significa que es opcional (puede ser null)
  materia      String      @default("General")
  fechaHora    DateTime    @default(now())
  completada   Boolean     @default(false)
  creadaEn     DateTime    @default(now())
  actualizadaEn DateTime    @updatedAt // Se actualiza automáticamente

  // Relación con Usuario: cada sesión pertenece a un usuario
  // En SQL esto se llama clave foránea (Foreign Key)
  usuarioId Int
  usuario    Usuario @relation(fields: [usuarioId], references: [id], onDelete: Cascade)
}

```

10.4 Crear las tablas en Supabase

`npx prisma db push` lee el `schema.prisma` y crea las tablas en Supabase automáticamente:

► CREAR TABLAS EN SUPABASE

```
npx prisma db push
```

► RESULTADO ESPERADO

```

Your database is now in sync with your Prisma schema.
✓ Generated Prisma Client (v5.x) to ./node_modules/@prisma/client

```

Verificar en Supabase: ir a tu proyecto en supabase.com → Table Editor. Deberías ver las tablas `usuarios` y `sesiones` creadas automáticamente.

💡 `npx prisma studio` abre una interfaz visual de la BD en el navegador (<http://localhost:5555>). Es como el `phpMyAdmin` pero mucho más moderno. Muy útil para verificar los datos durante el desarrollo.

10.5 Crear el cliente Prisma compartido

Crear `src/db.js` — un singleton de Prisma para no crear múltiples conexiones:

```

▶ src/db.js
// src/db.js
// Exporta una única instancia de PrismaClient
// (crear múltiples instancias puede causar errores de 'too many connections')
const { PrismaClient } = require('@prisma/client');

const prisma = new PrismaClient({
  log: ['error', 'warn'], // Solo logear errores y advertencias, no todas las queries
});

module.exports = prisma;

```

10.6 Migrar el controlador de sesiones a Prisma

Reemplazar completamente el contenido de `src/controllers/sesionesController.js`:

```

▶ REEMPLAZAR src/controllers/sesionesController.js
// src/controllers/sesionesController.js – CON PRISMA (reemplazar archivo completo)
const prisma = require('../db');
const { pub } = require('../redis/client');

// — GET /api/sesiones

const listar = async (req, res) => {
  // findMany: equivale a SELECT * FROM sesiones
  // include: hace un JOIN con la tabla usuarios para traer el nombre
  const sesiones = await prisma.sesion.findMany({
    include: { usuario: { select: { id: true, nombre: true, email: true } } },
    orderBy: { creadaEn: 'desc' } // Las más recientes primero
  });

  res.json({ ok: true, total: sesiones.length, datos: sesiones });
};

// — GET /api/sesiones/:id

const obtenerUna = async (req, res) => {
  const id = parseInt(req.params.id);
  const sesion = await prisma.sesion.findUnique({
    where: { id },
    include: { usuario: { select: { id: true, nombre: true } } }
  });

  if (!sesion) return res.status(404).json({ error: `Sesión ${id} no encontrada` });
  res.json(sesion);
};

// — POST /api/sesiones

const crear = async (req, res) => {
  const { titulo, descripcion, fechaHora, materia } = req.body;

  if (!titulo || titulo.trim() === '') {
    return res.status(400).json({ error: 'El campo titulo es obligatorio' });
  }

  // create: equivale a INSERT INTO sesiones (titulo, ...) VALUES (...)
  // req.usuario.id viene del middleware autenticar (el JWT decodificado)
  const sesion = await prisma.sesion.create({
    data: {

```



```

        titulo: titulo.trim(),
        descripcion: descripcion || '',
        materia: materia || 'General',
        fechaHora: fechaHora ? new Date(fechaHora) : new Date(),
        usuarioId: req.usuario.id // Asociar la sesión al usuario logueado
    },
    include: { usuario: { select: { nombre: true } } }
  });

  // Publicar evento Redis
  await pub.publish('study:sesion:creada', JSON.stringify({
    tipo: 'sesion:creada',
    payload: sesion,
    timestamp: new Date().toISOString()
  }));

  res.status(201).json(sesion);
};

// — PUT /api/sesiones/:id

const actualizar = async (req, res) => {
  const id = parseInt(req.params.id);

  // Verificar que la sesión existe Y pertenece al usuario logueado
  const sesionExistente = await prisma.sesion.findUnique({ where: { id } });
  if (!sesionExistente) return res.status(404).json({ error: 'Sesión no encontrada' });
  if (sesionExistente.usuarioId !== req.usuario.id) {
    return res.status(403).json({ error: 'No tienes permiso para modificar esta sesión' });
  }

  const { titulo, descripcion, materia, completada } = req.body;
  const sesion = await prisma.sesion.update({
    where: { id },
    data: { titulo, descripcion, materia, completada }
  });

  res.json(sesion);
};

// — DELETE /api/sesiones/:id

const eliminar = async (req, res) => {
  const id = parseInt(req.params.id);

  const sesionExistente = await prisma.sesion.findUnique({ where: { id } });
  if (!sesionExistente) return res.status(404).json({ error: 'Sesión no encontrada' });
  if (sesionExistente.usuarioId !== req.usuario.id) {
    return res.status(403).json({ error: 'No tienes permiso para eliminar esta sesión' });
  }

  await prisma.sesion.delete({ where: { id } });
  res.json({ ok: true, mensaje: `Sesión ${id} eliminada` });
};

module.exports = { listar, obtenerUna, crear, actualizar, eliminar };

```

10.7 Migrar auth a Prisma

Hacer lo mismo para las rutas de autenticación — modificar `src/routes/auth.js` para usar Prisma:

► ACTUALIZAR auth.js para usar Prisma

```
// En src/routes/auth.js – AGREGAR al inicio:
const prisma = require('../db');

// REEMPLAZAR la variable 'usuarios' en memoria por consultas Prisma:

// En POST /auth/register – reemplazar la lógica de usuarios[]:
const existente = await prisma.usuario.findUnique({ where: { email: email.toLowerCase() } });
if (existente) return res.status(409).json({ error: 'Ya existe una cuenta con ese email' });

const hash = await bcrypt.hash(password, 12);
const nuevoUsuario = await prisma.usuario.create({
  data: { nombre: nombre||'Usuario', email: email.toLowerCase(), password: hash }
});

res.status(201).json({
  mensaje: 'Cuenta creada exitosamente',
  usuario: { id: nuevoUsuario.id, nombre: nuevoUsuario.nombre, email: nuevoUsuario.email }
});

// En POST /auth/login – reemplazar el find() por:
const usuario = await prisma.usuario.findUnique({ where: { email: email.toLowerCase() } });
```

 Errores frecuentes con Prisma/Supabase

Error	Solución
P1001: Can't reach database server	Supabase pausó el proyecto después de 7 días sin uso. Ir a supabase.com → tu proyecto → clic en Restore.
migration failed: column doesn't exist	El schema.prisma cambió pero la BD no. Ejecutar: npx prisma db push --force-reset (solo en desarrollo, borra todos los datos).
Error: @prisma/client did not initialize	Ejecutar: npx prisma generate (regenera el cliente). Luego reiniciar el servidor.

✓ Al finalizar el Paso 10: tablas en Supabase. Crear sesión (autenticado) → persiste en la BD. Reiniciar el servidor → los datos siguen ahí. Verificar en Supabase Dashboard → Table Editor.

PASO 11 Swagger — Documentación Interactiva

Tiempo estimado: 45 minutos · El manual del sistema

🎯 **Swagger es como el catálogo de IKEA para tu API. En lugar de que el equipo de frontend te llame preguntando qué parámetros acepta cada endpoint, Swagger genera automáticamente una página web interactiva donde pueden ver y probar todo sin instalación. Es el estándar de la industria.**

11.1 Instalar Swagger

► INSTALAR SWAGGER

```
npm install swagger-ui-express swagger-jsdoc
```

11.2 Crear la configuración de Swagger

Crear src/swagger/config.js:

```
► src/swagger/config.js
// src/swagger/config.js
const swaggerJsdoc = require('swagger-jsdoc');

const opciones = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'StudySync API',
      version: '1.0.0',
      description: 'API para coordinación de grupos de estudio con notificaciones en tiempo
real',
      contact: { name: 'M.Sc. Jimmy Nataniel Requena Llorentty', email: 'docente@upds.edu' }
    },
    servers: [
      { url: 'http://localhost:3000', description: 'Desarrollo local' },
      { url: 'https://studysync-api.onrender.com', description: 'Producción' }
    ],
    components: {
      securitySchemes: {
        // Define cómo se envía el token JWT en los headers
        // Los endpoints que usan 'security: [{BearerAuth:[]}]' requieren token
        BearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' }
      }
    }
  },
  // Lee los comentarios JSDoc de todos los archivos en src/routes/
  apis: ['./src/routes/*.js']
};

module.exports = swaggerJsdoc(opciones);
```

11.3 Montar Swagger en app.js

Agregar en src/app.js ANTES del error handler global:

► AGREGAR EN src/app.js

```
// src/app.js – AGREGAR antes del error handler (el de 4 parámetros)
const swaggerUi = require('swagger-ui-express');
const swaggerSpec = require('./swagger/config');
```

```
// /api-docs muestra la interfaz Swagger en el navegador
// Esta ruta DEBE estar antes del error handler para que funcione
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec, {
  customSiteTitle: 'StudySync API Docs',
  swaggerOptions: { persistAuthorization: true } // Guardar el token entre recargas
}));
```

11.4 Documentar los endpoints con JSDoc

Abrir `src/routes/sesiones.js` y agregar comentarios JSDoc antes de cada ruta:

```
► DOCUMENTAR routes/sesiones.js
// src/routes/sesiones.js – CON documentación Swagger
const express = require('express');
const router = express.Router();
const ctrl = require('../controllers/sesionesController');
const autenticar = require('../middlewares/autenticar');

/**
 * @swagger
 * /api/sesiones:
 *   get:
 *     summary: Listar todas las sesiones de estudio
 *     tags: [Sesiones]
 *     responses:
 *       200:
 *         description: Lista de sesiones
 *         content:
 *           application/json:
 *             example:
 *               ok: true
 *               total: 2
 *               datos: [{id: 1, titulo: "Redis Pub/Sub", materia: "Prog IV"}]
 */
router.get('/', ctrl.listar);

/**
 * @swagger
 * /api/sesiones:
 *   post:
 *     summary: Crear una nueva sesión (requiere login)
 *     tags: [Sesiones]
 *     security:
 *       - BearerAuth: []
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required: [titulo]
 *             properties:
 *               titulo:
 *                 type: string
 *                 example: "Sesión de Redis Pub/Sub"
 *               materia:
 *                 type: string
 *                 example: "Programación IV"
 *             descripcion:
```

```
*           type: string
*   responses:
*     201:
*       description: Sesión creada exitosamente
*     400:
*       description: Faltan campos obligatorios
*     401:
*       description: Token JWT no proporcionado o inválido
*/
router.post('/', autenticar, ctrl.crear);

// ... (agregar comentarios similares para PUT y DELETE)
```

Verificar que /api-docs funciona: con el servidor corriendo, abrir en el navegador:
<http://localhost:3000/api-docs>

► **RESULTADO ESPERADO**

Página web de Swagger UI cargada. Muestra todos los endpoints documentados.
Aparece el botón "Authorize" en la esquina superior derecha.

💡 **Para probar endpoints protegidos desde Swagger: clic en 'Authorize' → ingresar el token (SIN 'Bearer', Swagger lo agrega automáticamente) → clic en Authorize → cerrar el modal → ahora los endpoints con candado usan el token.**

PASO 12 Seguridad Adicional — CORS, Rate Limiting, Helmet

Tiempo estimado: 30 minutos · Los guardias del sistema

🔒 **CORS es como el portero del edificio: controla quién puede entrar según su dirección de origen. Rate Limiting es como el límite de intentos en un cajero automático: después de cierto número de peticiones, te bloquea temporalmente. Helmet es como el escudo del servidor: agrega protecciones automáticas contra ataques web conocidos.**

12.1 Instalar dependencias de seguridad

▶ INSTALAR SEGURIDAD

```
npm install cors express-rate-limit helmet
```

12.2 Configurar en app.js

Abrir src/app.js y agregar ANTES de las rutas, en el bloque de middlewares:

▶ AGREGAR EN src/app.js

```
// src/app.js – AGREGAR en la sección de middlewares globales
const cors = require('cors');
const rateLimit = require('express-rate-limit');
const helmet = require('helmet');

// 1. HELMET: agrega ~12 cabeceras HTTP de seguridad automáticamente
// Previene: clickjacking, MIME sniffing, content injection, etc.
app.use(helmet());

// 2. CORS: controla qué dominios pueden hacer peticiones a la API
// origin: '*' significa cualquier dominio (OK para desarrollo)
// En producción: restringir al dominio del frontend
app.use(cors({
  origin: process.env.CORS_ORIGIN || '*',
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));

// 3. RATE LIMITING: máximo 100 peticiones por IP cada 15 minutos
// Previene ataques de fuerza bruta y DDoS básicos
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutos en milisegundos
  max: 100, // máximo 100 peticiones por IP en ese período
  message: { error: 'Demasiadas peticiones. Espera 15 minutos e intenta nuevamente.' },
  standardHeaders: true // Incluir cabeceras RateLimit-* en la respuesta
});
app.use('/api/', limiter); // Solo aplicar a rutas /api/ (no a /auth/ ni a /api-docs)
```

📝 El orden de los middlewares en Express importa. Siempre poner helmet, cors y rateLimit ANTES de las definiciones de rutas (app.use('/api/...')). Si se ponen después, no afectan a esas rutas.

12.3 Verificar el rate limiting

Para probar que funciona, hacer más de 100 peticiones rápidas. En Thunder Client, usar la opción 'Runner' para enviar múltiples peticiones. La petición 101 debe responder:

▶ RESULTADO ESPERADO

```
HTTP 429 Too Many Requests
```

```
{  
  "error": "Demasiadas peticiones. Espera 15 minutos e intenta nuevamente."  
}
```

✓ Al finalizar el Paso 12: Helmet activo (verificar cabeceras X-Frame-Options en la respuesta). CORS configurado. Rate limiting devuelve 429 después de 100 peticiones rápidas.

PASO 13 CloudFormation + LocalStack — Infraestructura como Código

Tiempo estimado: 1 hora · Los planos del sistema

🎯 Imagina que para abrir un segundo local de tu restaurante, en lugar de llamar a un arquitecto y explicarle todo desde cero, le das los planos del primer local y él construye una copia exacta. ¡aC son esos planos para la infraestructura de software. LocalStack simula AWS en tu computadora para que puedas practicar sin gastar dinero.

13.1 Instalar LocalStack con Docker

🖥️ Abrir PowerShell (con Docker Desktop ya corriendo en la barra de tareas):

▶ INICIAR LOCALSTACK

```
# Iniciar LocalStack en segundo plano (-d = detached)
docker run --rm -d -p 4566:4566 -p 4510-4559:4510-4559 localstack/localstack

# Verificar que está corriendo (tarda ~15 segundos en iniciar)
curl http://localhost:4566/_localstack/health
```

▶ RESULTADO ESPERADO

```
{"status":"running","services":{"..."}}
```

13.2 Configurar AWS CLI para LocalStack

▶ CONFIGURAR AWS CLI

```
# Instalar herramienta de CLI para LocalStack
pip install awscli-local

# Configurar credenciales "falsas" (LocalStack acepta cualquier valor)
aws configure --profile localstack
```

Cuando pregunte los valores, ingresar exactamente:

▶ VALORES DE CONFIGURACIÓN

```
AWS Access Key ID: test
AWS Secret Access Key: test
Default region name: us-east-1
Default output format: json
```

13.3 Crear la plantilla CloudFormation

Crear la carpeta cloudformation y el archivo template.yaml:

▶ CREAR ARCHIVO

```
mkdir cloudformation
type nul > cloudformation\template.yaml
```

Abrir cloudformation/template.yaml y escribir:

```
> cloudformation/template.yaml
AWSTemplateFormatVersion: '2010-09-09'
Description: >
  StudySync Infraestructura – Programación IV UPDS 2026
```


Recursos: S3 Bucket + DynamoDB Table + SSM Parameter

Parameters:

NombreProyecto:
Type: String
Default: studysync
Description: Nombre del proyecto en minúsculas
AllowedPattern: "[a-z0-9-]+"

Ambiente:

Type: String
Default: dev
AllowedValues: [dev, staging, prod]
Description: Ambiente de despliegue

Resources:

S3 Bucket para archivos estáticos (imágenes, documentos de estudio)

BucketArchivos:
Type: AWS::S3::Bucket
Properties:
BucketName: !Sub '\${NombreProyecto}-\${Ambiente}-archivos'
Tags:
- Key: Proyecto
Value: !Ref NombreProyecto
- Key: Ambiente
Value: !Ref Ambiente

Tabla DynamoDB para datos de sesiones (alternativa NoSQL a Supabase)

TablaSesiones:
Type: AWS::DynamoDB::Table
Properties:
TableName: !Sub '\${NombreProyecto}-\${Ambiente}-sesiones'
BillingMode: PAY_PER_REQUEST
AttributeDefinitions:
- AttributeName: id
AttributeType: S
KeySchema:
- AttributeName: id
KeyType: HASH

Parámetro SSM: URL pública de la API en producción

ParametroApiUrl:
Type: AWS::SSM::Parameter
DependsOn: TablaSesiones
Properties:
Name: !Sub '/\${NombreProyecto}/\${Ambiente}/api-url'
Type: String
Value: 'https://studysync-api.onrender.com'

Outputs:

NombreBucket:
Description: Nombre del bucket S3 creado
Value: !Ref BucketArchivos
Export:
Name: !Sub '\${NombreProyecto}-\${Ambiente}-BucketName'

NombreTabla:
Description: Nombre de la tabla DynamoDB creada
Value: !Ref TablaSesiones

13.4 Desplegar el stack en LocalStack

► CREAR STACK

```
# Crear el stack (crear todos los recursos descritos en el template)
awslocal cloudformation create-stack --stack-name studysync-stack --template-body
file://cloudformation/template.yaml --parameters
ParameterKey=NombreProyecto,ParameterValue=studysync
ParameterKey=Ambiente,ParameterValue=dev
```

► VERIFICAR ESTADO

```
# Verificar el estado del stack
awslocal cloudformation describe-stacks --stack-name studysync-stack --query
'Stacks[0].StackStatus'
```

► RESULTADO ESPERADO

```
"CREATE_COMPLETE"
```

► VER RECURSOS CREADOS

```
# Ver los recursos que se crearon
awslocal s3 ls
awslocal dynamodb list-tables

# Ver los outputs del stack
awslocal cloudformation describe-stacks --stack-name studysync-stack --query
'Stacks[0].Outputs' --output table
```

🐛 Errores en CloudFormation/LocalStack

Error	Solución
awslocal: command not found	pip install awscli-local. Si pip no funciona: python -m pip install awscli-local
YAML: mapping values not allowed	Error de indentado YAML. Usar solo espacios (nunca tabs). Instalar extensión YAML de Red Hat en VS Code.
Stack en ROLLBACK_COMPLETE	Algún recurso falló. Ver: awslocal cloudformation describe-stack-events --stack-name studysync-stack --query 'StackEvents[?ResourceStatus=='CREATE_FAILED']'

✅ Al finalizar el Paso 13: template.yaml con 3 recursos + 1 parámetro + 1 output. awslocal create-stack termina en CREATE_COMPLETE. awslocal s3 ls muestra el bucket. README actualizado con instrucciones de deploy.

PASO 14 QA — Control de Calidad del Sistema

Tiempo estimado: 2 horas · No omitir — los bugs aparecen aquí

🔗 QA es como la inspección de calidad antes de que el restaurante abra al público. El cocinero prueba cada plato: ¿sabe como debe saber? ¿la temperatura está bien? ¿la presentación es correcta? En software: ¿cada endpoint responde lo que debe responder? ¿los errores se muestran bien? ¿la seguridad funciona?

14.1 Checklist de QA funcional

Probar cada ítem en Thunder Client apuntando a <http://localhost:3000> (antes del deploy):

☑	Prueba	Esperado	Por qué importa
☐	GET /api/sesiones	200 + array	El endpoint principal funciona
☐	POST /auth/register con datos válidos	201 + usuario sin password	El hash funciona, no devuelve contraseña
☐	POST /auth/login con contraseña correcta	200 + token JWT	El flujo de auth funciona completo
☐	POST /api/sesiones SIN token	401 Unauthorized	El middleware autenticar funciona
☐	POST /api/sesiones CON token válido	201 + sesión creada	El flujo autenticado funciona
☐	GET /api/sesiones después de crear	200 + lista con datos	Prisma/Supabase persiste
☐	GET /api/sesiones/999 (ID inexistente)	404 Not Found	El manejo de errores funciona
☐	POST /api/sesiones sin campo titulo	400 Bad Request	La validación de datos funciona
☐	POST /auth/login contraseña incorrecta	401 Unauthorized	Bcrypt compare funciona
☐	Abrir /api-docs en navegador	Swagger UI visible	La documentación está disponible
☐	GET /api/sesiones tras reiniciar servidor	200 + datos anteriores	Supabase persiste correctamente

14.2 QA de tiempo real

5. Abrir 2 pestañas del navegador en <http://localhost:3000>. Ambas deben mostrar '● Conectado'.
6. Con Thunder Client: POST /api/sesiones con token válido.
7. Verificar que AMBAS pestañas del navegador muestran el nuevo evento sin recargar.
8. Verificar en Supabase Dashboard → Table Editor que la sesión aparece en la tabla.

14.3 QA de seguridad

- ▶ Hacer 101 peticiones rápidas GET /api/sesiones → la petición 101 debe devolver 429.
- ▶ Probar con token expirado (cambiar JWT_EXPIRES_IN=1s en .env, esperar 2 segundos, probar) → debe devolver 401 con 'Token expirado'.
- ▶ Verificar que las cabeceras de respuesta incluyen X-Frame-Options y X-Content-Type-Options (Helmet activo).

14.4 Registrar y corregir bugs

Crear un archivo BUGS.md en la raíz del proyecto para registrar cada bug encontrado y cómo se resolvió. Esto sirve como evidencia del proceso de QA:

```
► BUGS.md
# BUGS.md — Registro de bugs encontrados en QA

## Bug #001
- **Fecha**: 27 May 2026
- **Descripción**: POST /api/sesiones devolvía 500 cuando fechaHora tenía formato incorrecto
- **Causa**: new Date('fecha-invalida') devuelve NaN en lugar de lanzar un error
- **Solución**: Agregar validación: if (isNaN(new Date(fechaHora))) return res.status(400)...
- **Estado**:  Resuelto
```

PASO 15 Deploy Final — StudySync en Producción

Tiempo estimado: 45 minutos · El sistema vive en internet

15.1 Preparar para producción

```

▶ COMMIT FINAL
# Verificar que .env NO está en el repo
git status
# Si aparece .env en la lista, ejecutar:
# git rm --cached .env
# git commit -m "fix: remover .env del tracking"

# Commit final con todo el código
git add .
git commit -m "feat: StudySync MVP completo - API+Redis+JWT+Supabase+Swagger+IaC"
git push origin main

```

15.2 Configurar variables de entorno en Render

Ir a render.com → tu Web Service studysync-api → clic en 'Environment':

Variable	Valor (de tu .env local)
PORT	10000 (Render asigna el puerto — usar 10000 explícito)
NODE_ENV	production
REDIS_URL	redis://default:TuPass@tu-host.upstash.io:6379 (copiar de Upstash)
DATABASE_URL	postgresql://postgres:TuPass@db.XXXX.supabase.co:5432/postgres (copiar de Supabase)
JWT_SECRET	Un string largo y aleatorio. En producción cambiar el que tienes en .env local.
JWT_EXPIRES_IN	1h (1 hora es el estándar para sesiones activas)

● **NUNCA** copiar el archivo .env completo a Render. Cada variable se agrega individualmente. Si accidentalmente subes credenciales al repositorio GitHub, cambiarlas INMEDIATAMENTE en Upstash y Supabase desde sus dashboards.

15.3 Disparar el deploy

Render detecta automáticamente el push a main y despliega. También puedes forzar el deploy desde el dashboard: clic en 'Manual Deploy' → 'Deploy latest commit'.

```

▶ VERIFICAR PRODUCCIÓN
# Verificar el deploy desde PowerShell o curl
curl https://studysync-api.onrender.com/

```

```

▶ RESULTADO ESPERADO
{"mensaje":"StudySync API funcionando","version":"1.0.0",...}

```

15.4 Verificación final en producción

Cambiar la URL en Thunder Client de `http://localhost:3000` a tu URL de Render y repetir las pruebas principales:

- ▶ GET `https://studysync-api.onrender.com/api/sesiones` → 200
- ▶ POST `https://studysync-api.onrender.com/auth/register` → 201
- ▶ POST `https://studysync-api.onrender.com/auth/login` → 200 + token
- ▶ Abrir `https://studysync-api.onrender.com/api-docs` en el navegador → Swagger cargado

Errores frecuentes al pasar a producción

Error	Solución
Service unavailable / 502 Bad Gateway	El servidor se cayó. Ver Logs en el dashboard de Render. Causa más común: variable de entorno faltante.
P1001: Can't reach database en producción	Supabase pausó el proyecto. Ir a <code>supabase.com</code> → Restore. En producción considerar usar la URL del Connection Pooler de Supabase.
Socket.io no conecta en Render free	Render free no garantiza WebSockets persistentes. Solución: usar transports: ['polling'] en el cliente Socket.io. O migrar a Railway.

PASO 16 README Final y Preparación de la Demo

Tiempo estimado: 1 hora · La presentación del sistema

16.1 Actualizar el README.md

El README es lo primero que ve el docente al abrir el repositorio. Debe ser claro, profesional y tener la URL de producción.

```

▶ README.md
# StudySync API – Sistema de Coordinación de Grupos de Estudio

## Descripción
StudySync es una API REST distribuida para coordinar sesiones de estudio universitarias
con notificaciones en tiempo real. Resuelve el problema de la desorganización académica
entre estudiantes.

## URL de Producción
**API:** https://studysync-api.onrender.com
**Documentación Swagger:** https://studysync-api.onrender.com/api-docs

## Tecnologías
| Tecnología | Uso en el sistema |
|-----|-----|
| Node.js + Express | API REST con 8 endpoints |
| Upstash Redis | Pub/Sub para notificaciones en tiempo real |
| Socket.io | WebSockets para actualización de la UI |
| JWT + bcrypt | Autenticación y hashing de contraseñas |
| Supabase + Prisma | PostgreSQL en la nube como base de datos |
| Swagger/OpenAPI | Documentación interactiva de la API |
| AWS CloudFormation | Infraestructura como Código (LocalStack) |

## Arquitectura
...
Cliente Web → API REST (Express) → Supabase (PostgreSQL)
                ↓
                Upstash Redis (Pub/Sub)
                ↓
                Socket.io → Navegador (tiempo real)
...

## Cómo ejecutar localmente
```bash
git clone https://github.com/TUUSUARIO/studysync-api.git
cd studysync-api
npm install
Crear .env con REDIS_URL, DATABASE_URL, JWT_SECRET
npm run dev
Abrir http://localhost:3000
```

## Endpoints principales
Método	Ruta	Auth	Descripción
GET	/api/sesiones	No	Listar sesiones
POST	/api/sesiones	Sí	Crear sesión
POST	/auth/register	No	Registrar usuario
POST	/auth/login	No	Login + token JWT

```

16.2 Guión de la demo de 10 minutos

Preparar las pestañas del navegador ANTES de que empiece el turno:

| Tiempo | Etapas | Qué mostrar | Pestaña del navegador |
|--------------|------------------------|---|-------------------------------------|
| 0:00 – 1:30 | El problema | Explicar en 90 segundos: los grupos de estudio coordinados por WhatsApp son caóticos. StudySync centraliza y notifica en tiempo real. | Ninguna — hablar directamente |
| 1:30 – 4:00 | Demo API | Thunder Client: register → login → copiar token → GET sesiones (401) → GET sesiones con token (200) → POST crear sesión (201). | Thunder Client + Supabase Dashboard |
| 4:00 – 6:30 | Redis en vivo | Mostrar 2 pestañas del navegador abiertas. Crear sesión en Thunder Client. Las 2 pestañas se actualizan solas. El suscriptor imprime en consola. | 2 pestañas en / + terminal |
| 6:30 – 8:30 | Swagger + Seg. | Abrir /api-docs. Mostrar todos los endpoints. Authorize con el token. Probar GET /api/sesiones desde Swagger. Mostrar 401 sin token. | /api-docs en navegador |
| 8:30 – 10:00 | GitHub + Cierre | Mostrar el repositorio: historial de commits, múltiples miembros. Cerrar: 'StudySync resuelve X problema real con las 4 tecnologías del curso, desplegado en producción'. | GitHub repo en navegador |



Ensayar la demo 2 veces el día anterior. En el primer ensayo van a encontrar al menos 1 problema técnico. Mejor encontrarlo en privado que en la sustentación.

Apéndice — Los 20 Errores Más Comunes y Sus Soluciones

Errores de Node.js y Express

| Error | Solución |
|------------------------------|---|
| Cannot find module 'X' | npm install X — ejecutar en la carpeta raíz del proyecto (donde está package.json) |
| req.body is undefined | Falta app.use(express.json()). Agregarlo ANTES de las rutas en app.js. |
| EADDRINUSE: :::3000 | Puerto ocupado. Cambiar PORT=3001 en .env o cerrar la otra terminal con el servidor. |
| router.get is not a function | Exportaste el router incorrectamente. Verificar: module.exports = router; al final del archivo. |
| res.json() called twice | Falta el 'return' antes de res.status(404).json(). Agregar: return res.status(404).json(...) |

Errores de Redis / Upstash

| Error | Solución |
|----------------------------------|--|
| ECONNREFUSED 127.0.0.1:6379 | REDIS_URL en .env apunta a localhost. Debe apuntar a Upstash: redis://default:PASS@HOST:PORT |
| max daily commands reached | Límite de 10,000 comandos/día alcanzado. Crear segunda BD en Upstash o esperar reset a medianoche UTC. |
| Pub sin Sub (mensajes no llegan) | Publicador y suscriptor usan la misma conexión Redis. Deben ser objetos separados. Ver src/redis/client.js |
| CERTIFICATE_VERIFY_FAILED | Solo en Python. En Node.js no ocurre. Si estás en Python: redis.from_url(url, ssl_cert_reqs=None) |

Errores de JWT y Autenticación

| Error | Solución |
|-----------------------|--|
| invalid signature | JWT_SECRET cambió. Hacer logout + nuevo login para obtener token con el secreto actual. |
| jwt malformed | El token fue copiado incompleto. Un JWT tiene exactamente 3 partes separadas por puntos. |
| TokenExpiredError | El token venció. Hacer login nuevamente para obtener uno nuevo. |
| 401 incluso con token | Falta 'Bearer ' antes del token. Header debe ser: Authorization: Bearer eyJhbG... |

Errores de Prisma y Supabase

| Error | Solución |
|-------|----------|
|-------|----------|

| | |
|---------------------------------------|---|
| P1001: Can't reach database | Supabase pausó el proyecto. Ir a supabase.com → Restore (tarda ~60 segundos). |
| migration failed | Schema cambió pero BD no. Ejecutar: npx prisma db push --force-reset (borra datos de desarrollo). |
| @prisma/client not initialized | Ejecutar: npx prisma generate. Luego reiniciar el servidor. |
| Unique constraint failed | Email duplicado en registro. La ruta register ya valida esto — verificar que funciona el 409. |

StudySync en Tiempo Récord — Guía Práctica Completa · UPDS 2026

M.Sc. Jimmy Nataniel Requena Llorentty

Universidad Privada Domingo Savio — Facultad de Ingeniería — Ingeniería de Sistemas